# CẨM NANG NGÀNH DEVOPS

Giáo trình kỹ thuật · bản đọc thử Chương 1

Chưa phải bản phát hành hoàn chỉnh

# MỤC LỤC

Chương 1. Từ chương trình đến tiến trình 3

1.1. Ba lớp thường bị gọi chung là “ứng dụng” . . . . . 3

1.2. Một tiến trình ra đời và đổi chương trình . . . . . 3

1.3. Tên tệp, file descriptor và đối tượng đang mở . . . . . 4

1.4. Quyền và giới hạn: tiến trình không có toàn quyền chỉ vì nó chạy . . . . . 4

1.5. Từ tiến trình đến dịch vụ . . . . . 5

1.6. Áp mô hình vào một sự cố nhỏ . . . . . 5

Bài tập . . . . . 5

Nguồn chương . . . . . 6

Lời giải 7

Chương 1. Từ chương trình đến tiến trình . . . . . 7

# Chương 1. Từ chương trình đến tiến trình

Một dịch vụ có thể có mã nguồn đúng mà vẫn không phục vụ được yêu cầu. Tệp thực thi có thể tồn tại, nhưng chưa có tiến trình nào đang chạy; tiến trình có thể chạy, nhưng không mở được cổng; cổng có thể mở, nhưng tiến trình không đọc được tệp cấu hình. Để chẩn đoán những tình huống này, trước hết cần biết hệ điều hành đã biến chương trình thành một thực thể đang hoạt động như thế nào. Chương này xây dựng mô hình ấy trước khi nói đến container, điều phối hay giám sát. [B2]

## 1.1. Ba lớp thường bị gọi chung là “ứng dụng”

Mã nguồn là văn bản mô tả hành vi. Tệp thực thi hoặc chương trình được thông dịch là vật liệu để khởi chạy. Tiến trình là một lần thực thi cụ thể: nó có định danh, không gian địa chỉ, trạng thái xử lý, thông tin định danh người dùng và tập tài nguyên đang mở. Một chương trình có thể được chạy nhiều lần, tạo ra nhiều tiến trình độc lập. Ngược lại, một tiến trình có thể thay chương trình đang thực thi mà vẫn giữ định danh tiến trình, khi dùng cơ chế `execve`. [R2]

Sự phân biệt này giải thích vì sao “build thành công” không đồng nghĩa “dịch vụ sống”. Build xác nhận một số tính chất của artifact; nó không chứng minh hệ điều hành cấp đủ tài nguyên, địa chỉ mạng sẵn có hay cấu hình runtime hợp lệ. Đối với người vận hành, câu hỏi cần tách làm hai: artifact nào được đưa tới máy, và tiến trình nào đang thực thi artifact đó? Nếu hai câu trả lời khác nhau, một lần triển khai có thể đã thay tệp trên đĩa mà chưa thay phiên bản đang phục vụ.

CPU thực thi lệnh của tiến trình theo các khoảng thời gian; bộ nhớ chứa mã và dữ liệu cần cho lần chạy; nhân hệ điều hành quản lý tài nguyên và cung cấp các lời gọi hệ thống. Đó là mô hình khởi đầu, chưa phải toàn bộ chi tiết của Linux. Trong một tiến trình còn có thể có nhiều luồng thực thi cùng chia sẻ phần lớn không gian địa chỉ. Vì thế, số tiến trình không phải số công việc đồng thời, và mức dùng CPU không thể suy ra chỉ từ số PID.

## 1.2. Một tiến trình ra đời và đổi chương trình

Trên Linux, `fork` tạo tiến trình con. Tiến trình cha và con nhận các giá trị trả về khác nhau, nhờ đó có thể đi theo hai nhánh xử lý. Không nên hình dung rằng toàn bộ bộ nhớ vật lý bị sao chép ngay lập tức: triển khai Linux dùng các trang copy-on-write, tạo bản riêng khi cần ghi. Đây là chi tiết triển khai Linux, không phải định nghĩa phổ quát của mọi hệ điều hành. [R1]

Sau bước tạo con, một cách khởi chạy chương trình khác là gọi `execve`. Lời gọi này thay process image hiện tại bằng chương trình mới; nó không tạo thêm một tiến trình mới. Nếu thành công, luồng lệnh cũ không tiếp tục ngay sau lời gọi. Nếu lỗi, lời gọi trả về và chương trình cũ còn cơ hội xử lý. Cặp cơ chế tạo con rồi thay chương trình giúp hiểu vì sao một shell có thể tiếp tục tồn tại sau khi khởi chạy lệnh khác, đồng thời vì sao tiến trình của lệnh có PID riêng. [R1][R2]

Mô hình này cũng giải thích một lỗi vận hành phổ biến: chương trình con kế thừa tài nguyên mà người gọi quên đóng. Theo mặc định, một file descriptor có thể còn mở qua `execve`; cờ

close-on-exec thay đổi hành vi đó. Nếu vô tình giữ đầu ghi của một pipe ở tiến trình khác, đầu đọc có thể chưa nhìn thấy kết thúc luồng dù bên tạo dữ liệu đã xong. Kết luận này là hệ quả của vòng đời tài nguyên, không phải một quy tắc riêng của Bash. [R2][R3]

## 1.3. Tên tệp, file descriptor và đối tượng đang mở

Khi tiến trình gọi `open`, nó nhận một số nguyên nhỏ không âm: file descriptor. Số này là chỉ số trong bảng descriptor của chính tiến trình. Descriptor không phải tên tệp, cũng không phải bản thân nội dung tệp. Nó tham chiếu tới một open file description do nhân quản lý, nơi giữ vị trí đọc/ghi hiện tại và các cờ trạng thái. Hai descriptor có thể cùng tham chiếu một open file description; khi đó chúng chia sẻ vị trí đọc/ghi. [R3]

Sự khác biệt có ý nghĩa khi đọc log. Nếu một tiến trình đã mở tệp log và người vận hành đổi tên hoặc xóa đường dẫn, descriptor đang mở vẫn có thể trỏ tới đối tượng cũ. Tạo một tệp mới ở cùng tên đường dẫn không buộc tiến trình chuyển đầu ghi sang tệp mới. Phần thực hành về log rotation ở các chương sau sẽ dựa trên quan hệ này để giải thích vì sao có trường hợp dung lượng đĩa chưa được giải phóng và vì sao cần yêu cầu dịch vụ mở lại log. [R3]

Ba descriptor quen thuộc khi khởi chạy chương trình là đầu vào chuẩn, đầu ra chuẩn và đầu lỗi chuẩn. Chuyển hướng trong shell thay đổi các luồng mà tiến trình nhận được; nó không sửa ý nghĩa của chương trình. Khi một dịch vụ ghi thông báo lỗi vào đầu ra chuẩn, người thu thập log vẫn có thể lấy được dữ liệu, nhưng việc phân loại mức độ lỗi trở nên khó hơn. Do vậy, quy ước đầu ra là một phần của hợp đồng vận hành.

## 1.4. Quyền và giới hạn: tiến trình không có toàn quyền chỉ vì nó chạy

Một tiến trình mang thông tin định danh và chịu kiểm tra quyền khi truy cập tài nguyên. Có quyền đọc một đường dẫn không đồng nghĩa có quyền thực thi tệp, và có quyền thực thi một tệp không đồng nghĩa được đọc mọi tệp mà chương trình sẽ mở sau đó. Nhìn từ bên ngoài, cả hai tình huống có thể hiện thành "ứng dụng không khởi động"; bên trong, điểm lỗi nằm ở những bước khác nhau. Về sau, mô hình này sẽ nối với quyền Linux, IAM và quyền của tài khoản dịch vụ trong Kubernetes.

Tiến trình còn hoạt động trong các giới hạn tài nguyên. Một lần tạo tiến trình con có thể thất bại khi đạt giới hạn số tác vụ; `fork` báo lỗi thay vì bảo đảm thành công. Vì vậy, một bộ điều phối không thể chữa một chương trình tạo tiến trình vô hạn chỉ bằng cách tự khởi động lại nó. Tự phục hồi là chính sách phản ứng với lỗi; ngăn lỗi lặp lại đòi sửa nguyên nhân hoặc đặt giới hạn thích hợp. [R1]

Lưu ý rằng quyền truy cập và khả năng nhìn thấy tài nguyên là hai câu hỏi khác nhau. Một namespace có thể cho tiến trình một góc nhìn riêng về một loại tài nguyên của hệ thống. Nhưng góc nhìn riêng không tự chứng minh rằng tiến trình an toàn hay không thể tác động đến thứ bên ngoài ranh giới đó. Ở phần container, ta sẽ kết hợp namespace với giới hạn tài nguyên, quyền và chính sách bảo mật thay vì xem "cách ly" là một thuộc tính nhị phân. [R4]

## 1.5. Từ tiến trình đến dịch vụ

Dịch vụ là một hợp đồng vận hành đặt lên tiến trình: khởi động khi nào, cấu hình từ đâu, báo trạng thái ra sao, dừng theo tín hiệu nào, có khởi động lại không và dữ liệu bền vững nằm ở đâu. Không phải mọi tiến trình đều là dịch vụ. Một công cụ chạy xong rồi thoát có vòng đời khác một HTTP server tồn tại lâu và tiếp nhận yêu cầu mới.

Khi một tiến trình thoát, hệ điều hành có thể ghi nhận mã kết thúc. Mã này cho biết chương trình kết thúc theo quy ước nào; tự nó chưa cho biết nguyên nhân gốc. Cùng một mã khác không có bảo đảm diễn giải giống nhau giữa mọi chương trình. Một hệ thống giám sát tốt cần xem thêm thời điểm, stderr, tình trạng tài nguyên và sự kiện từ bộ quản lý dịch vụ. Đọc mỗi trạng thái "đã dừng" rồi suy ra lỗi mạng, lỗi quyền hay thiếu bộ nhớ là bước nhảy thiếu chứng cứ.

Một dịch vụ cũng có thể còn tiến trình nhưng không còn khả năng làm việc có ích. Vòng lặp bị kẹt, hàng đợi đầy hoặc phụ thuộc phía sau không phản hồi đều có thể khiến PID vẫn tồn tại. Vì vậy, giám sát sự sống của tiến trình và kiểm tra khả năng phục vụ là hai phép đo khác nhau. Ta sẽ gặp lại khác biệt này khi phân tích healthcheck của Docker, probe của Kubernetes và SLI của SRE.

## 1.6. Áp mô hình vào một sự cố nhỏ

Giả sử sau triển khai, trình duyệt báo không kết nối được tới dịch vụ. Cách suy nghĩ theo lớp bắt đầu từ artifact và tiến trình: bản build đã đến máy chưa; tiến trình có được tạo không; nó còn tồn tại hay đã thoát? Nếu đã thoát, cần xem mã kết thúc và thông báo lỗi. Nếu còn tồn tại, xem nó đã tạo socket lắng nghe đúng địa chỉ và cổng chưa. Sau đó mới xem đường đi mạng và phụ thuộc ở xa. Trật tự này không bảo đảm tìm ra mọi lỗi, nhưng ngăn việc sửa DNS khi chương trình thực ra chưa mở cổng.

Khi kiểm tra, phải phân biệt quan sát với kết luận. "Không thấy PID trong danh sách" là quan sát tại thời điểm đo; "build lỗi" chỉ là một giả thuyết. "Có PID" cũng chưa chứng minh dịch vụ khỏe. Một hồ sơ sự cố tốt ghi mốc thời gian, lệnh hoặc nguồn quan sát, artifact đang chạy, cấu hình có hiệu lực và thay đổi vừa xảy ra. Nhờ thế, người khác có thể phản biện kết luận hoặc tái dựng chuỗi nhân quả thay vì tin vào trí nhớ của người trực hệ thống.

Chương này không yêu cầu thuộc lòng mọi lời gọi hệ thống. Điều cần giữ là ba câu hỏi: thứ gì tồn tại trên đĩa, tiến trình nào đang chạy, và tiến trình ấy đang nắm những tài nguyên nào? Phần sau sẽ dùng cùng ba câu hỏi khi đi từ một tiến trình trên máy tới container, rồi tới workload trong cụm.

## Bài tập

C1.1. Một bản triển khai thay tệp thực thi trên đĩa nhưng không khởi động lại dịch vụ. Giải thích vì sao không được kết luận tiến trình đang phục vụ đã chạy phiên bản mới. Nêu phép kiểm tra cần làm.

C1.2. Một chương trình vẫn ghi vào tệp log đã bị đổi tên; tệp mới cùng tên vẫn rỗng. Hãy dùng quan hệ giữa đường dẫn, descriptor và open file description để giải thích. Đề xuất một cách xử lý an toàn.

C1.3. `fork` thành công, `execve` thất bại. Trong mô hình của chương này, có bao nhiêu tiến trình sau `fork`, và tiến trình nào có thể nhận biết lỗi `execve`? Vì sao không được coi `execve` là thao tác tạo tiến trình?

## Nguồn chương

[B2] OSTEP, v1.10: https://pages.cs.wisc.edu/~remzi/OSTEP/

[R1] Linux man-pages, fork(2): https://man7.org/linux/man-pages/man2/fork.2.html

[R2] Linux man-pages, execve(2): https://man7.org/linux/man-pages/man2/execve.2.html

[R3] Linux man-pages, open(2): https://man7.org/linux/man-pages/man2/open.2.html

[R4] Linux man-pages, namespaces(7): https://man7.org/linux/man-pages/man7/namespaces.7.html

# Lời giải

## Chương 1. Từ chương trình đến tiến trình

C1.1. Artifact trên đĩa và lần thực thi đang chạy là hai thực thể khác nhau. Việc ghi đè một đường dẫn không chứng minh process image đã đổi. Cần kiểm tra PID và thời điểm khởi động, lệnh và artifact mà dịch vụ đang dùng, rồi so với định danh bản triển khai. Nếu ứng dụng có endpoint phiên bản, chỉ dùng nó khi biết endpoint phản ánh đúng build đang chạy. Khởi động lại phải theo quy trình triển khai, có kiểm tra sức khỏe và đường quay lui.

C1.2. Descriptor của tiến trình tham chiếu open file description đang mở, không tra cứu lại tên đường dẫn cho mỗi lần ghi. Đổi tên hoặc tạo tệp mới cùng tên không chuyển descriptor cũ. Cách xử lý là yêu cầu dịch vụ mở lại log bằng cơ chế được nó hỗ trợ, hoặc khởi động lại có kiểm soát sau khi bảo đảm log đã được thu thập và khả năng phục vụ không bị gián đoạn ngoài mức chấp nhận. Không xóa cưỡng bức dữ liệu trước khi xác định descriptor và nhu cầu lưu giữ.

C1.3. Sau `fork` thành công có hai tiến trình: cha và con. Nếu con gọi `execve` và lời gọi thất bại, chính nhánh con nhận giá trị lỗi và có thể báo lại; cha chỉ biết nếu có cơ chế thông báo hoặc chờ kết thúc. `execve` thay chương trình trong tiến trình gọi, nên không tăng số tiến trình. Trên đường thành công, lệnh sau `execve` trong chương trình cũ không chạy tiếp.